

Micro-Transformer: A From-Scratch Autograd and Attention Engine for Character-Level Language Modeling

Team Members: Manikanta Harsha Nadimpalli, Luis Blanco Hoyos

Emails (SUNet): nmharsha@stanford.edu, blancol@stanford.edu

1 Motivation

We are building a reverse-mode autograd engine and a small transformer [1] on top of it from scratch in NumPy, with one specific scientific question in mind: *how do LayerNorm [4] placement (pre-norm, post-norm, none) and residual connections shape the gradient signal that reaches the early layers of a decoder-only transformer during training on character-level language modeling?* The from-scratch build is the instrument; the gradient-flow ablation is the contribution. Existing frameworks expose hooks and functional Jacobians, but constructing the engine ourselves gives us conceptual clarity over every chain-rule edge and exact, low-overhead access to per-layer activation and gradient statistics, with no framework internals to reverse-engineer.

The milestone deliverables described below are the foundation of that instrument: a data pipeline, three classical baselines, a scalar reverse-mode engine, a tensor reverse-mode engine with the full op surface needed for an MLP, and a first non-trivial language model trained end-to-end through our own engine. The ablation grid itself is post-milestone work and tracked under *Next Steps*.

2 Methods

Data and tokenization. Tiny Shakespeare [3] (1,115,394 characters, 65 unique symbols), char-level tokenization. Train/val split is 90/10 *by position*, not random; random char-level shuffling would leak local context across the boundary and inflate val performance. Sampling uses caller-supplied `np.random.Generator` so every run is reproducible at the seed.

Baselines (closed-form and SGD). Three reference points anchor every later result:

- **Uniform predictor.** Cross-entropy floor of $\log V$ nats.
- **Closed-form bigram** with Laplace smoothing ($\alpha=1$). Direct count $\rightarrow$ normalize on train. Smoothing is required because val contains bigrams unseen in train.
- **Softmax regression by SGD.** The same model class as the bigram (logits = $W[i] + b$) trained by mini-batch SGD with hand-derived gradients in plain NumPy. Convergence behavior under SGD is itself a sanity check: the SGD trajectory should approach the closed-form CE.

Scalar reverse-mode autograd. A Karpathy-style [2] `Value` class wrapping a Python float. Eight elementary ops (`+`, `*`, `**`, `relu`, `tanh`, `exp`, `log`, `neg/sub/div` as compositions). Each forward op installs a closure that captures (`self`, `other`, `out`) and pushes the upstream gradient through its local Jacobian; `backward()` topologically sorts from the output and walks in reverse. Verified against (i) central differences and (ii) torch.autograd to bit-identical agreement on a randomized two-layer neuron.

Tensor reverse-mode autograd. A NumPy-backed `Tensor` class lifting the same closure pattern to batched arrays. The two genuine complications are broadcasting in backward (collapsed by an `unbroadcast` helper that sums along broadcast leading axes and size-1 dims) and lazy grad allocation. The op surface is sufficient for the MLP and forward-compatible with attention: elementwise binary/unary with broadcasting, reductions `sum/mean` (axis + keepdims) with grad re-broadcast, batched matmul (`@`) with leading-batch fan-in, integer-array gather (the embedding lookup, with `np.add.at` scatter-add backward), reshape, `relu`, `tanh`, numerically stable `log_softmax`, and a fused `cross_entropy(logits, targets)` whose backward is the closed-form $(\text{softmax} - \text{onehot})/B$. Every op has a numerical-gradient-check test (the non-negotiable Week 2 deliverable from our project plan).

First model: char-MLP via our engine. A Bengio-style [5] character-MLP: previous $T=3$ characters $\rightarrow$ embedding $\mathbb{R}^{V \times d} \rightarrow$ flatten $\rightarrow$ $\tanh$ -hidden $\rightarrow$ linear $\rightarrow$ cross-entropy on the next character. 8,401 trainable parameters; trained end-to-end with our autograd, no torch in the loop.

Closed-form attention backward (parallel work, Week-4 spec). A complete derivation of the scaled-dot-product attention backward — softmax-row Jacobian collapse, batched-matmul fan-in, three projection paths summing into X , causal-mask handling — together with a PyTorch reference implementation and four parity tests at `float64`, `1e-9` tolerance against `torch.autograd.grad`. This is the spec we will port into the Tensor engine in the next phase.

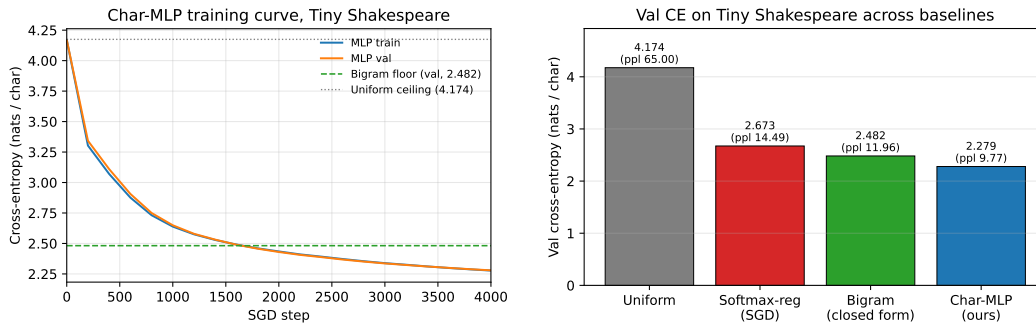
3 Preliminary Experiments and Results

Headline. Table 1 reports validation cross-entropy on Tiny Shakespeare across all four models. The MLP, trained end-to-end through our autograd engine, beats the closed-form bigram floor by 0.20 nats (0.29 bpc, an 18% reduction in perplexity).

Model	Val CE (nats)	Val bpc	Val perplexity
Uniform	4.1744	6.0224	65.00
Softmax regression (SGD, 3k steps)	2.6733	3.8568	14.49
Bigram (closed form, $\alpha = 1$)	2.4819	3.5806	11.96
Char-MLP via our autograd	2.2795	3.2886	9.77

Table 1: Validation cross-entropy on Tiny Shakespeare (111,539 chars). Lower is better; bpc = nats / $\ln 2$. The MLP uses 3-character context and 8,401 parameters; trained for 4,000 SGD steps in 14 seconds (numpy-only, single CPU).

Why the MLP wins, and by how much we expected. The bigram is the optimal 2-gram model for this data; SGD on the same parameterization (table softmax-regression) cannot beat it asymptotically. The MLP sits in a strictly larger hypothesis class — it conditions on 3 characters of context and learns a non-linear feature combination via the $\tanh$ hidden layer. A 0.20-nat val improvement is in line with what 3-gram-style models achieve on this corpus and confirms the engine’s gradients are propagating useful learning signal, not just numerically correct noise.



(a) MLP train+val CE vs. SGD step. Val crosses the bigram floor (dashed green) around step 1700. (b) Val CE across the four reference points. Lower is better.

Figure 1: Headline results from the engine and baselines.

Engine validation. Correctness, not raw performance, is the immediate concern: an undetected gradient bug invalidates every later experiment. We layered three checks:

- **Numerical gradient check** on every op individually (*add*, *mul*, *div*, *pow*, broadcast cases, reductions on every keepdims/axis combo, matmul on three batching patterns, gather with repeated indices, reshape, tanh, relu, log_softmax, cross_entropy). Tolerance `atol=1e-7`, `rtol=1e-5`; all 59 cases pass.
- **End-to-end gradcheck on the MLP forward pass**: gradient of the CE loss against all five parameter tensors (E , W_1 , b_1 , W_2 , b_2) on randomized inputs.
- **PyTorch parity**: same MLP forward and backward computed in our engine and in `torch.autograd`; loss bit-identical, all five gradients within 5×10^{-17} — below float64 machine epsilon ($\sim 2.2 \times 10^{-16}$).

The total project test suite has 103 passing cases.

What the eval pipeline gives us for free. Because `full_corpus_ce` runs over every consecutive bigram pair (rather than a sampled subset), val CE is directly comparable across model classes — bigram counts, SGD-trained logits, and MLP forward all consume the same sequence and report the same mean over the same 111,539 positions. The -0.20 -nat gap in Table 1 reflects model class, not evaluation noise.

4 Next Steps

1. **Port Luis’s closed-form attention backward into the Tensor engine.** The derivation, PyTorch reference, and $1e-9$ -tolerance parity tests already exist (committed under `tests/reference_attention.py`); the engine has matmul, log-softmax, and a gather-shaped scatter, which is everything needed. Estimated 2–3 days.
2. **LayerNorm forward + backward**, validated by gradcheck and torch parity.
3. **Decoder block** with toggleable LayerNorm placement (pre-norm / post-norm / none) and residual on/off — exactly the 6 configurations the ablation grid will sweep.
4. **Ablation grid.** Train all 6 configs on Tiny Shakespeare, single seed, identical batches (the data layer’s reproducible RNG was built for this). Per-step log: layer-wise $\|g\|_2$, activation norms, and the singular-value spectrum of layer Jacobians.
5. **Comparison curves**: gradient-norm trajectories per config, training curves per config, val perplexity at convergence.

The MLP result here is a sanity floor; the transformer ablation is the contribution.

5 Team Contributions

- **Manikanta Harsha Nadimpalli**: Tiny Shakespeare data pipeline (loader, tokenizer, batched sampler with reproducibility); three classical baselines (uniform, closed-form Laplace bigram, softmax-regression by SGD with hand-derived gradients); scalar reverse-mode autograd engine with numerical gradient check + PyTorch parity; tensor reverse-mode autograd engine (broadcasting, reductions, matmul, gather, reshape, activations, log-softmax, cross-entropy) with per-op numerical gradient checks and end-to-end PyTorch parity; char-MLP forward and training loop; experiment scripts, results JSONs, figures; this writeup.
- **Luis Blanco Hoyos**: Closed-form derivation of the scaled-dot-product attention backward (softmax-row Jacobian collapse, batched-matmul fan-in, three-path X contribution, causal-mask handling) in `writeups/cs229/attention_backward.md`; PyTorch reference forward and closed-form backward in `tests/reference_attention.py`; four float64 parity tests at $1e-9$ tolerance against `torch.autograd`, with and without causal mask. This is the exact specification the Week-4 attention port will be built and tested against.

References

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I. (2017). Attention is all you need. *NeurIPS* 30.

- [2] Karpathy, A. (2020). micrograd: A tiny scalar-valued autograd engine. <https://github.com/karpathy/micrograd>.
- [3] Karpathy, A. (2015). The Unreasonable Effectiveness of Recurrent Neural Networks. <https://karpathy.github.io/2015/05/21/rnn-effectiveness/>.
- [4] Ba, J.L., Kiros, J.R., Hinton, G.E. (2016). Layer Normalization. *arXiv:1607.06450*.
- [5] Bengio, Y., Ducharme, R., Vincent, P., Jauvin, C. (2003). A Neural Probabilistic Language Model. *JMLR* 3, 1137–1155.